Module 11 Lab Session 2: JWT Bearer Authentication & Refresh Token Rotation

Session	2 of 3
Exercises	Exercise **3** & Exercise **4**
Integrity Lab tier	**Tier 2** — JWT claim claims, HMAC-SHA256 signature, AddJwtBearer authentication, RefreshToken entity, rotation, single-use validation

Story thread

When a user submits valid credentials to `/api/auth/login`, the API must issue a cryptographically signed JSON Web Token (JWT) containing user identity claims (`sub`, `email`, `role`). Short-lived access tokens (15 minutes) reduce the impact of token theft. To maintain continuous user sessions, you will implement **Refresh Token Rotation**: every refresh call invalidates the old token (`IsUsed = true`) and issues a brand-new pair. Submitting an already-used refresh token triggers **token theft detection** and revokes all active sessions for that user.

Exercise 3: JWT Generation & Bearer Authentication Pipeline

Step 1: Configure Jwt Settings & User Secrets

In `appsettings.Development.json`:

```
{
  "Jwt": {
    "Issuer": "https://localhost:5001",
    "Audience": "tms-client",
    "ExpiryMinutes": 15
  }
}
```

Store the cryptographic signing key safely in User Secrets:

```
dotnet user-secrets set "Jwt:Key" "A-Very-Long-Secret-Key-For-TMS-Auth-Stored-Safely-2026"
```

Step 2: Implement TokenService

Create `TmsApi.Infrastructure/Services/TokenService.cs`:

```
namespace TmsApi.Infrastructure.Services;

public class TokenService
{
    private readonly IConfiguration _config;

    public TokenService(IConfiguration config)
    {
        _config = config;
    }

    public string GenerateJwt(TmsUser user, IList<string> roles)
    {
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.NameIdentifier, user.Id),
            new Claim(ClaimTypes.Email, user.Email ?? string.Empty),
            new Claim("FirstName", user.FirstName)
        };

        foreach (var role in roles)
        {
            claims.Add(new Claim(ClaimTypes.Role, role));
        }

        var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
        var creds = new SigningCredentials(key,
SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires:
DateTime.UtcNow.AddMinutes(int.Parse(_config["Jwt:ExpiryMinutes"]!)),
            signingCredentials: creds
        );

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

Step 3: Configure Authentication Pipeline in Program.cs

In `Program.cs`, add JWT Bearer authentication and register `TokenService`:

```
builder.Services.AddScoped<TokenService>();

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme =
JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme =
JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    };
});
```

Exercise 4: Refresh Token Rotation & Theft Detection

Step 1: Create RefreshToken Entity

Create `TmsApi.Domain.Entities/RefreshToken.cs`:

```
namespace TmsApi.Domain.Entities;

public class RefreshToken
{
    public int Id { get; set; }
    public string Token { get; set; } = string.Empty;
    public string UserId { get; set; } = string.Empty;
    public DateTime ExpiresAt { get; set; }
    public bool IsUsed { get; set; }
    public bool IsRevoked { get; set; }
}
```

Add `DbSet<RefreshToken> RefreshTokens { get; set; }` to `TmsDbContext.cs` and run migration:

```
dotnet ef migrations add AddRefreshTokens --project TmsApi.Infrastructu
re/TmsApi.Infrastructure.csproj --startup-project TmsApi.Api/TmsApi.Api.
csproj

dotnet ef database update --project TmsApi.Infrastructure/TmsApi.Infras
tructure.csproj --startup-project TmsApi.Api/TmsApi.Api.csproj
```

Step 2: Update AuthController with Token Generation & Rotation

Update `Controllers/AuthController.cs` to inject `TmsDbContext` and `TokenService`, and add `POST /api/auth/refresh`:

```
namespace TmsApi.Api.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly UserManager<TmsUser> _userManager;
    private readonly RoleManager<IdentityRole> _roleManager;
    private readonly TmsDbContext _context;
    private readonly TokenService _tokenService;

    public AuthController(
        UserManager<TmsUser> userManager,
        RoleManager<IdentityRole> roleManager,
        TmsDbContext context,
        TokenService tokenService)
    {
        _userManager = userManager;
        _roleManager = roleManager;
        _context = context;
        _tokenService = tokenService;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginRequest
request)
    {
        var user = await _userManager.FindByEmailAsync(request.Email);
        if (user == null) return Unauthorized(new { detail = "Invalid
credentials." });

        if (await _userManager.IsLockedOutAsync(user))
        {
```

```
            return StatusCode(423, new { detail = "Account locked due
to multiple failed login attempts." });
        }

        var validPassword = await _userManager.CheckPasswordAsync(user,
request.Password);
        if (!validPassword)
        {
            await _userManager.AccessFailedAsync(user);
            return Unauthorized(new { detail = "Invalid
credentials." });
        }

        await _userManager.ResetAccessFailedCountAsync(user);

        var roles = await _userManager.GetRolesAsync(user);
        var accessToken = _tokenService.GenerateJwt(user, roles);

        // Issue initial Refresh Token
        var refreshToken = new RefreshToken
        {
            Token = Guid.NewGuid().ToString("N"),
            UserId = user.Id,
            ExpiresAt = DateTime.UtcNow.AddDays(7),
            IsUsed = false,
            IsRevoked = false
        };

        _context.RefreshTokens.Add(refreshToken);
        await _context.SaveChangesAsync();

        return Ok(new
        {
            accessToken,
            refreshToken = refreshToken.Token
        });
    }

    public record RefreshRequest(string RefreshToken);

    [HttpPost("refresh")]
    public async Task<IActionResult> Refresh([FromBody] RefreshRequest
request)
    {
        var storedToken = await _context.RefreshTokens
            .FirstOrDefaultAsync(rt => rt.Token ==
request.RefreshToken);
```

```
        if (storedToken == null)
        {
            return Unauthorized(new { detail = "Invalid refresh
token." });
        }

        // Theft Detection: If an ALREADY-USED token is submitted,
revoke ALL tokens for this user!
        if (storedToken.IsUsed)
        {
            var userTokens = await _context.RefreshTokens
                .Where(rt => rt.UserId == storedToken.UserId)
                .ToListAsync();

            foreach (var t in userTokens)
            {
                t.IsRevoked = true;
            }

            await _context.SaveChangesAsync();
            return Unauthorized(new { detail = "Token theft detected.
All user sessions revoked." });
        }

        if (storedToken.IsRevoked || storedToken.ExpiresAt <
DateTime.UtcNow)
        {
            return Unauthorized(new { detail = "Refresh token expired
or revoked." });
        }

        // Mark current token as used
        storedToken.IsUsed = true;

        // Issue brand-new Refresh Token pair
        var newRefreshToken = new RefreshToken
        {
            Token = Guid.NewGuid().ToString("N"),
            UserId = storedToken.UserId,
            ExpiresAt = DateTime.UtcNow.AddDays(7),
            IsUsed = false,
            IsRevoked = false
        };

        _context.RefreshTokens.Add(newRefreshToken);
        await _context.SaveChangesAsync();

        var user = await _userManager.FindByIdAsync(storedToken.UserId);
```

```
        var roles = await _userManager.GetRolesAsync(user!);
        var newAccessToken = _tokenService.GenerateJwt(user!, roles);

        return Ok(new
        {
            accessToken = newAccessToken,
            refreshToken = newRefreshToken.Token
        });
    }
}
```

Client Integration Note (Token Migration from M10): Unlike M10's cookie-only transport where the browser implicitly received `Set-Cookie`, `POST /api/auth/login` and `POST /api/auth/refresh` now return `{ accessToken, refreshToken }` explicitly in the JSON response payload. In **Session 3 (Exercise 6)**, we update Angular's `AuthService` to hold `accessToken` in memory and configure `jwtInterceptor` to attach it as `Authorization: Bearer <token>` to outgoing API requests.

Verification Checkpoint (Session 2)

1. **Login & Claims Inspection:**
   - Send `POST /api/auth/login`. Confirm response returns `accessToken` and `refreshToken`.
   - Copy `accessToken` to `https://jwt.ms`. Verify claims: `sub`, `email`, `role`, and `exp` (15 min).
2. **Refresh Token Rotation Check:**
   - Send `POST /api/auth/refresh` with `refreshToken`. Confirm a new `accessToken` and new `refreshToken` are returned.
3. **Theft Revocation Test:**
   - Resend the *old* (now used) refresh token to `POST /api/auth/refresh`.
   - Confirm `401 Unauthorized` ("Token theft detected. All user sessions revoked.").
   - Verify in database that all `RefreshTokens` for that user have `IsRevoked = 1`.